

Exercise 7 - Diffusion Generation of MNIST digits

Erik Stolt

What you did and how

In this exercise we explore Diffusion Models, a relatively simple architecture that exploits noise for data augmentation and to generate new samples following the same distribution as the training data. In its simplest form, we start by, in time steps, adding noise to a training data point (this is the forward pass), then we train a network to denoise the the training example in order to reconstruct the image. In the end, the idea is that we can simply start with an image that is complete noise (Gaussian in our case, but it could in theory be another distribution), and then pass it through the network to get a new unique image that does not appear in the data set. In practice we do this by following the algorithm [1]:

Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on

$$\nabla_{\theta} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta} \left(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t \right) \right\|^2$$

- 6: **until** converged
-

where $\mathbf{x}_0$ is an image from the training data set. The advantage of diffusion models is that a single training example can be used multiple times, since in each iteration we add an independent noise vector to it. Notice that $\boldsymbol{\epsilon}_{\theta}$ is actually the total noise added to the image over t time steps. This is not what we want, we want the model to predict the noise added to the image in each time step. It turns out, as done in the paper, that one can parametrize the model using a change of variable to obtain this. The end result is given in algorithm 2 [1] where we now have an additional scaling factor in front of $\boldsymbol{\epsilon}_{\theta}$:

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

We can now sample from algorithm 2 as many times as we want to end up with a distribution that follows the training data, and which we can input random noise $\mathbf{x}_T$ to and receive a new image.

However, the difference in this exercise compared to *Exercise 6 - Simple Diffusion*, is that now we have an "actual" image with 28×28 pixels. We also use the more complex network architecture, UNET. Furthermore, training is sped up by introducing **SAMPLING TIMESTEP** which means when do not go in timesteps of 1 in the sampling stage, we actually take larger steps with minimal sacrifice of image quality. This is done to speed up the training process [2].

What results you obtained

One example print-out during training looked like this:

Epoch 18/30 100%| 422/422 [2:39 < 00:00, 2,65it/s] Train Loss = 0.0372 | Val Loss = 0.0369
sampling loop time step: 100%| 250/250 [00:06<00:00, 38.70it/s]

And the following plots were obtained:

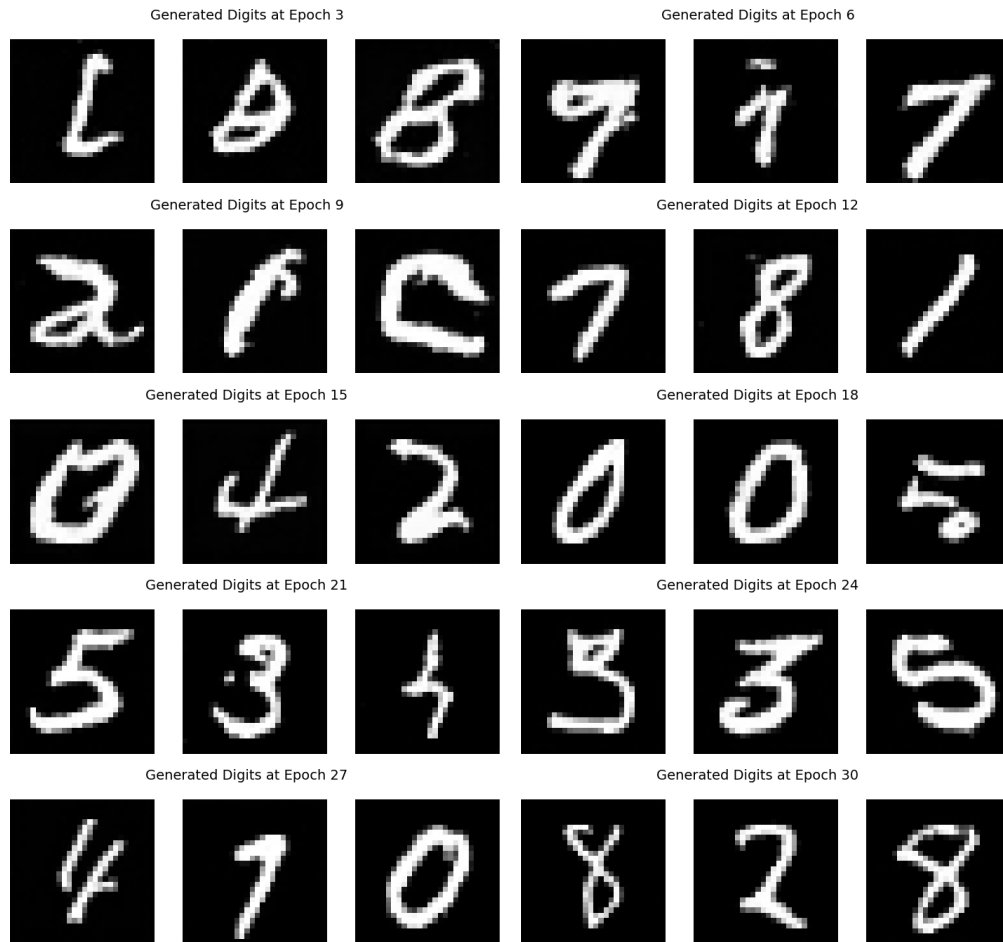


Figure 1: Evolution of generated handwritten digits over 30 epochs.

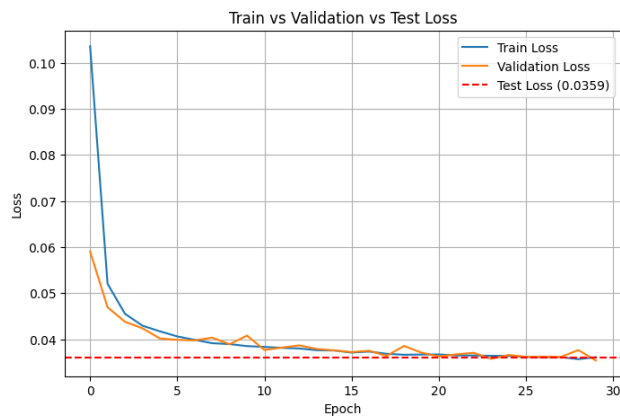


Figure 2: Loss as a function of the epoch.

Where the digits at epoch 30 looks really good. Also the loss curve seems to converge at around 15 epochs for this model architecture. Overall, the results are as expected. We see that the evolution of the digits gets better. We also make the comparison to *Exercise 5: Generative Adversarial Networks* where we also generated digits but using GANs. The result using a UNET is instantly better and is more stable in training, in Exercise 5 we had problems with white spots in the image that added an unnatural look. However, these are almost completely gone and we obtained a very realistic handwritten digits that cannot be distinguished to not be in the data set.

What challenges you encountered and what could be improved

The main problem I encountered in this exercise was the training time. Even one epoch took a while to train, so I decided to train for 30 epochs as it gave a good trade-off between convergence and training time. The resulting images between the earlier epochs and the final epochs are not that larger either, which further justifies the fewer epoch training. This leaves us with an improvement opportunity, mainly, to train for more epochs and with potentially larger network if one has the compute to do this. Also one can implement early stopping and other schedulers to stabilize training. I did not do this in this exercise since its not the main part of the exercise and it is not needed when we train for so few epochs.

Link to GitHub

Link: <https://github.com/erst6955/Advanced-Applied-Deep-Learning-in-Physics-And-Engineering>

Code (for backup)

~\Desktop\Advanced Applied Deep Learning In Physics And Engineering\Exercises\Exercise 7 -
Diffusion Generation of MNIST digists\Diffusion MNIST.py

```
1 import torchvision
2 # For image transforms
3 from torchvision import transforms
4 # For DATA SET
5 import torchvision.datasets as datasets
6 # For Pytorch methods
7 import torch
8 import torch.nn as nn
9 # For Optimizer
10 import torch.optim as optim
11 # FOR DATA LOADER
12 from torch.utils.data import DataLoader, random_split
13 import os
14 # MODEL
15 from denoising_diffusion_pytorch import Unet, GaussianDiffusion
16 from tqdm import tqdm
17 # FOR PLOTTING
18 import matplotlib.pyplot as plt
19
20
21 # Hyperparameters
22 LEARNING_RATE = 4e-4
23 BATCH_SIZE = 128
24 N_EPOCHS = 30
25 IMAGE_SIZE = 28
26 TIME_STEPS = 1000
27 SAMPLING_TIMESTEPS = 250
28 NUM_EXAMPLES = 3
29
30 # we define a tranform that converts the image to tensor
31 myTransforms = transforms.Compose([transforms.ToTensor()]) # maps [0,255] → [0.0, 1.0]
32
33 # Full MNIST training set
34 full_train_dataset = datasets.MNIST(root="dataset/", train=True, transform=myTransforms,
35 download=True)
36
37 # 90% training, 10% validation
38 train_size = int(0.9 * len(full_train_dataset)) # 54,000
39 val_size = len(full_train_dataset) - train_size # 6,000
40 train_dataset, val_dataset = random_split(full_train_dataset, [train_size, val_size])
41
42 # Loaders
43 train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
44 val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
45
46 # Test set stays as it is
```

```

46 test_dataset = datasets.MNIST(root='dataset/', train=False, download=False,
47 transform=myTransforms)
48
49 # ===== Training =====
50 def train_model():
51     train_losses, val_losses = [], []
52
53     for epoch in range(N_EPOCHS):
54         model.train()
55         total_train_loss = 0
56         for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{N_EPOCHS}"):
57             images, _ = batch
58             images = images.to(device)
59
60             loss = diffusion(images) # Pass through UNET
61             optim.zero_grad()
62             loss.backward()
63             optim.step()
64
65             total_train_loss += loss.item()
66
67         avg_train_loss = total_train_loss / len(train_loader)
68         train_losses.append(avg_train_loss)
69
70         # Validation
71         model.eval()
72         total_val_loss = 0
73         with torch.no_grad():
74             for batch in val_loader:
75                 images, _ = batch
76                 images = images.to(device)
77                 val_loss = diffusion(images)
78                 total_val_loss += val_loss.item()
79         avg_val_loss = total_val_loss / len(val_loader)
80         val_losses.append(avg_val_loss)
81
82         print(f"Epoch {epoch+1} | Train Loss = {avg_train_loss:.4f} | Val Loss =
83 {avg_val_loss:.4f}")
84
85         if (epoch + 1) % 3 == 0 or epoch == N_EPOCHS - 1:
86             sampled_images = diffusion.sample(batch_size=3) # Sample images
87             save_images(sampled_images, epoch + 1) # Save images
88
89     return train_losses, val_losses
90
91 def save_images(images, epoch):
92     images = images.cpu().detach()
93

```

```

94     fig, axs = plt.subplots(1, 3, figsize=(9, 3))
95     for i in range(3):
96         axs[i].imshow(images[i][0], cmap="gray")
97         axs[i].axis("off")
98
99     fig.suptitle(f"Generated Digits at Epoch {epoch}", fontsize=14)
100
101     os.makedirs("Figures", exist_ok=True)
102     plt.savefig(f"Figures/{epoch}.png", bbox_inches="tight")
103     plt.close()
104
105 # ===== Evaluation =====
106 def compute_test_loss():
107     model.eval()
108     total_test_loss = 0
109     with torch.no_grad():
110         for batch in test_loader:
111             images, _ = batch
112             images = images.to(device)
113             loss = diffusion(images)
114             total_test_loss += loss.item()
115     return total_test_loss / len(test_loader)
116
117
118
119 # ===== Plotting =====
120
121 def plot_losses(train_losses, val_losses, test_loss):
122     plt.figure(figsize=(8, 5))
123     plt.plot(train_losses, label="Train Loss")
124     plt.plot(val_losses, label="Validation Loss")
125     plt.axhline(y=test_loss, color='r', linestyle='--', label=f"Test Loss ({test_loss:.4f})")
126     plt.xlabel("Epoch")
127     plt.ylabel("Loss")
128     plt.title("Train vs Validation vs Test Loss")
129     plt.legend()
130     plt.grid(True)
131     plt.savefig("Figures/loss_curve.png", bbox_inches="tight")
132     plt.close()
133
134
135 # ===== RUN CODE =====
136
137 DIM = 32 # base dimensionality (number of channels) for the U-Net model.
138 DIM_MULTS = (1, 2, 5) # UNET with layers and channel widths; L1: 32x1, L2: 32x2, L3: 32x5.
139 # Output dimension is implicit and same as input dim (makes sense since its a diffusion model)
140 model = Unet(dim = DIM, dim_mults = DIM_MULTS, flash_attn = False, channels = 1) # defines
141 # the model
142 # Set up diffusion model with gaussian noise
143 diffusion = GaussianDiffusion(model, image_size = IMAGE_SIZE, timesteps = TIME_STEPS,
144                               sampling_timesteps = SAMPLING_TIMESTEPS) # number of sampling timesteps (using ddim for

```

```
    faster inference [see ddim paper])
142
143 optim = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
144 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
145 print(f"Using device: {device}")
146 model.to(device)
147 diffusion.to(device)
148
149 train_losses, val_losses = train_model()
150 test_loss = compute_test_loss()
151 plot_losses(train_losses, val_losses, test_loss)
152
153
```

References

- [1] Tom B Brown et al. “Language Models are Few-Shot Learners”. In: *arXiv preprint arXiv:2006.11239* (2020). URL: <https://arxiv.org/abs/2006.11239>.
- [2] Jiaming Song, Chenlin Meng, and Stefano Ermon. “Denoising Diffusion Implicit Models”. In: *arXiv preprint arXiv:2010.02502* (2020). URL: <https://arxiv.org/abs/2010.02502>.